

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3

По дисциплине: «Модели решения задач в интеллектуальных системах»
Тема: «Многослойный персептрон для XOR с ReLU на скрытых слоях: сравнение
активационных функций»

Выполнил:
Студент 4 курса
Группы ИИ-25
Трошин И.Д.
Проверила:
Андренко К.В.

Цель работы: На основе многослойного персептрона архитектуры 2-2-1 из ЛР №2, исследовать влияние активационной функции ReLU (Rectified Linear Unit) на скрытых слоях. Провести сравнительный анализ обучения сети с функцией активации Сигмоида (базовая конфигурация) и ReLU (новая конфигурация) по скорости сходимости, качеству обучения, форме разделяющей поверхности и устойчивости к инициализации весов.

Задачи лабораторной работы

1. Взять за основу код многослойного персептрона 2-2-1 с функцией потерь BCE из ЛР №2.
2. Реализовать активационную функцию ReLU на скрытом слое и осуществить корректное обратное распространение ошибки для ReLU. На выходном слое оставить Сигмоиду.
3. Обучить сеть на полной таблице истинности в двух конфигурациях:
 - **Конфигурация А (базовая):** MLP 2-2-1, Сигмоида на всех слоях, BCE, нормализованные целевые значения (0/1);
 - **Конфигурация В (новая):** MLP 2-2-1, ReLU на скрытом слое, Сигмоида на выходном, BCE, нормализованные (0/1).
4. Критерий остановки для обеих конфигураций: суммарная ошибка $E_s \leq E_e$ (как в ЛР №2).
5. Для каждой конфигурации зафиксировать:
 - Количество эпох до критерия остановки.
 - Итоговую суммарную ошибку и ассигасу.
 - Среднюю абсолютную ошибку в исходной шкале $[c_0; c_1]$.
6. Провести не менее 5 запусков обучения для каждой конфигурации с разными seed и сравнить устойчивость.
7. Построить графики:
 - График сходимости (суммарная ошибка E_s от эпохи) для обеих конфигураций на одних осях.
 - Диаграмма разброса числа эпох по 5 запускам (столбики) для конфигурации А и В.
8. Визуализация разделяющей поверхности:
 - Для представительного экземпляра каждой конфигурации построить карту (heatmap) выходного значения сети на плоскости $(A, B) \in [-10; 10]^2$ с наложением 4 точек обучающей выборки.
 - Описать, как ReLU изменяет форму разделяющей поверхности (кусочно-линейные границы vs гладкие).

9. Реализовать режим функционирования сети:

- Пользователь вводит (A, B) из $[-10; 10]$.
- Сеть выводит: выход в нормализованной шкале, выход в исходной шкале $[c0; c1]$, класс.
- Продемонстрировать на 4 обучающих примерах + 2–3 дополнительных парах.

10. Создать таблицу сравнения (статистика по 5 запускам, MAE для обеих конфигураций).

11. Вывод: ответить на все вопросы и провести анализ результатов.

Вариант 13

Вариант 13 ($c0 = 4, c1 = -1$)

A	B	$A \oplus B$
4	4	4
4	-1	-1
-1	4	-1
-1	-1	4

Ход работы

Описание изменений в архитектуре и алгоритме обучения по сравнению с ЛР №1: формула нормализации целевых значений, формула обратного пересчёта выхода, вывод градиента для ВСЕ + сигмоида.

Функции активации и нормализация:

```
def sigmoid(S):  
    return 1 / (1 + np.exp(-S))  
  
def sigmoid_derivative(S):  
    y = sigmoid(S)  
    return y * (1 - y)  
  
def ReLU(S):  
    return np.maximum(0, S)  
  
def ReLU_derivative(S):  
    return (S > 0).astype(float)  
  
def normalize(value, c0, c1):  
    return (value - c0) / (c1 - c0)  
  
def denormalize(value, c0, c1):  
    return c0 + value * (c1 - c0)  
  
def forward(x, W1, T1, W2, T2, activation):  
    S1 = np.dot(x, W1) - T1  
    h = activation(S1)  
    S2 = np.dot(h, W2) - T2  
    y = sigmoid(S2)  
    return S1, h, S2, y  
  
def predict(x, W1, T1, W2, T2, activation):  
    _, _, _, y = forward(x, W1, T1, W2, T2, activation)  
    return y
```

Обучение:

```
def train(X, e, activation, activation_derivative, alpha=0.9, Ee=0.01, max_epochs=10000, seed=42):

    np.random.seed(seed)

    W1 = np.random.uniform(-0.1, 0.1, (2, 2))
    T1 = np.random.uniform(-0.1, 0.1, 2)

    W2 = np.random.uniform(-0.1, 0.1, 2)
    T2 = np.random.uniform(-0.1, 0.1)

    errors = []
    epoch = 0

    while True:
        for xi, ei in zip(X, e):
            S1, h, S2, y = forward(xi, W1, T1, W2, T2, activation)

            delta2 = y - ei
            delta1 = (activation_derivative(S1) * W2 * delta2)

            W2 = W2 - alpha * h * delta2
            T2 = T2 + alpha * delta2
            W1 = W1 - alpha * np.outer(xi, delta1)
            T1 = T1 + alpha * delta1

        Es = 0
        for xi, ei in zip(X, e):
            y = predict(xi, W1, T1, W2, T2, activation)
            y = np.clip(y, 1e-10, 1 - 1e-10) #защита от log(0)
            Es += -(ei * np.log(y) + (1 - ei) * np.log(1 - y))
        errors.append(Es)
        epoch += 1

        if Es <= Ee or epoch >= max_epochs:
            break

    return W1, T1, W2, T2, errors
```

Результат

Random_seed = 42:

Исходные входы:

```
[[ 4  4]
 [ 4 -1]
 [-1  4]
 [-1 -1]]
```

Нормализованные входы:

```
[[ -0.  -0.]
 [ -0.   1.]
 [  1.  -0.]
 [  1.   1.]]
```

Исходные классы:

```
[ 4 -1 -1  4]
```

Нормализованные классы:

```
[-0.   1.   1. -0.]
```

КОНФИГУРАЦИЯ А — SIGMOID

Эпох обучения: 3974

Финальная ошибка: 0.009999524283952892

Ассурасу: 100.00%

Средняя абсолютная ошибка: 0.012483

Ответы сети:

Вход: [4. 4.], Ожидалось: 4.0000, Получено: 3.9898, Норм. выход: 0.0020

Вход: [4. -1.], Ожидалось: -1.0000, Получено: -0.9888, Норм. выход: 0.9978

Вход: [-1. 4.], Ожидалось: -1.0000, Получено: -0.9888, Норм. выход: 0.9978
Вход: [-1. -1.], Ожидалось: 4.0000, Получено: 3.9826, Норм. выход: 0.0035

КОНФИГУРАЦИЯ Б — ReLU

Эпох обучения: 10000

Финальная ошибка: 1.3890956565635162

Accurasy: 75.00%

Средняя абсолютная ошибка: 1.250040

Мертвые нейроны

Ответы сети:

Вход: [4. 4.], Ожидалось: 4.0000, Получено: 3.9999, Норм. выход: 0.0000

Вход: [4. -1.], Ожидалось: -1.0000, Получено: 1.6315, Норм. выход: 0.4737

Вход: [-1. 4.], Ожидалось: -1.0000, Получено: -0.9999, Норм. выход: 1.0000

Вход: [-1. -1.], Ожидалось: 4.0000, Получено: 1.6315, Норм. выход: 0.4737

Скрытый слой ReLU:

X=[-0. -0.], S1=[2.52380164e+00 -3.65900570e-06], h=[2.52380164 0.],
W2=[-4.26999489 3.90589938], S2=-10.8819, y=0.0000

X=[-0. 1.], S1=[-0.34020958 -3.43554816], h=[0. 0.], W2=[-4.26999489 3.90589938],
S2=-0.1053, y=0.4737

X=[1. -0.], S1=[-7.04655463e-06 2.90048600e+00], h=[0. 2.900486],
W2=[-4.26999489 3.90589938], S2=11.2237, y=1.0000

X=[1. 1.], S1=[-2.86401826 -0.53505849], h=[0. 0.], W2=[-4.26999489 3.90589938],
S2=-0.1053, y=0.4737

5 запусков Sigmoid

Seed=1: эпох=10000, ошибка=1.917195, accurasy=75.00%, MAE=1.707309, критерий=нет

Seed=2: эпох=10000, ошибка=1.917790, accurasy=75.00%, MAE=1.707668, критерий=нет

Seed=3: эпох=10000, ошибка=1.917475, accurasy=75.00%, MAE=1.707490, критерий=нет

Seed=4: эпох=3010, ошибка=0.009997, accurasy=100.00%, MAE=0.012480, критерий=да

Seed=5: эпох=3348, ошибка=0.009998, accurasy=100.00%, MAE=0.012481, критерий=да

5 запусков ReLU

Seed=1: эпох=10000, ошибка=1.389096, accurasy=75.00%, MAE=1.250041, критерий=нет

Seed=2: эпох=10000, ошибка=1.389093, accurasy=75.00%, MAE=1.250037, критерий=нет

Seed=3: эпох=10000, ошибка=2.772649, accurasy=50.00%, MAE=2.500000, критерий=нет

Seed=4: эпох=488, ошибка=0.009999, accurasy=100.00%, MAE=0.012477, критерий=да

Seed=5: эпох=10000, ошибка=1.916004, accurasy=75.00%, MAE=1.629092, критерий=нет

Проверка 4 точек XOR

A=4, B=4 -> y=0.0000, y_real=3.9999, класс=4

A=4, B=-1 -> y=0.4737, y_real=1.6315, класс=4

A=-1, B=4 -> y=1.0000, y_real=-0.9999, класс=-1

A=-1, B=-1 -> y=0.4737, y_real=1.6315, класс=4

Random_seed = 4:

Исходные входы:

```
[[ 4 4]
 [ 4 -1]
 [-1 4]
 [-1 -1]]
```

Нормализованные входы:

```
[[-0. -0.]
 [-0. 1.]
 [ 1. -0.]
 [ 1. 1.]]
```

Исходные классы:

[4 -1 -1 4]

Нормализованные классы:

[-0. 1. 1. -0.]

КОНФИГУРАЦИЯ А — SIGMOID

Эпох обучения: 3010

Финальная ошибка: 0.00999698295516324

Accuracy: 100.00%

Средняя абсолютная ошибка: 0.012480

Ответы сети:

Вход: [4. 4.], Ожидалось: 4.0000, Получено: 3.9894, Норм. выход: 0.0021

Вход: [4. -1.], Ожидалось: -1.0000, Получено: -0.9888, Норм. выход: 0.9978

Вход: [-1. 4.], Ожидалось: -1.0000, Получено: -0.9888, Норм. выход: 0.9978

Вход: [-1. -1.], Ожидалось: 4.0000, Получено: 3.9831, Норм. выход: 0.0034

КОНФИГУРАЦИЯ Б — ReLU

Эпох обучения: 488

Финальная ошибка: 0.009999232117240762

Accuracy: 100.00%

Средняя абсолютная ошибка: 0.012477

Ответы сети:

Вход: [4. 4.], Ожидалось: 4.0000, Получено: 3.9954, Норм. выход: 0.0009

Вход: [4. -1.], Ожидалось: -1.0000, Получено: -0.9796, Норм. выход: 0.9959

Вход: [-1. 4.], Ожидалось: -1.0000, Получено: -0.9796, Норм. выход: 0.9959

Вход: [-1. -1.], Ожидалось: 4.0000, Получено: 3.9955, Норм. выход: 0.0009

5 запусков Sigmoid

Seed=1: эпох=10000, ошибка=1.917195, accuracy=75.00%, MAE=1.707309, критерий=нет

Seed=2: эпох=10000, ошибка=1.917790, accuracy=75.00%, MAE=1.707668, критерий=нет

Seed=3: эпох=10000, ошибка=1.917475, accuracy=75.00%, MAE=1.707490, критерий=нет

Seed=4: эпох=3010, ошибка=0.009997, accuracy=100.00%, MAE=0.012480, критерий=да

Seed=5: эпох=3348, ошибка=0.009998, accuracy=100.00%, MAE=0.012481, критерий=да

5 запусков ReLU

Seed=1: эпох=10000, ошибка=1.389096, accuracy=75.00%, MAE=1.250041, критерий=нет

Seed=2: эпох=10000, ошибка=1.389093, accuracy=75.00%, MAE=1.250037, критерий=нет

Seed=3: эпох=10000, ошибка=2.772649, accuracy=50.00%, MAE=2.500000, критерий=нет

Seed=4: эпох=488, ошибка=0.009999, accuracy=100.00%, MAE=0.012477, критерий=да

Seed=5: эпох=10000, ошибка=1.916004, accuracy=75.00%, MAE=1.629092, критерий=нет

Проверка 4 точек XOR

A=4, B=4 -> y=0.0009, y_real=3.9954, класс=4

A=4, B=-1 -> y=0.9959, y_real=-0.9796, класс=-1

A=-1, B=4 -> y=0.9959, y_real=-0.9796, класс=-1

A=-1, B=-1 -> y=0.0009, y_real=3.9955, класс=4

Введите пару чисел (A, B): 5 5

Нормализованный выход: 0.0000

Выход в исходной шкале: 4.0000

Ближайший класс: 4

Введите пару чисел (A, B): 5 0

Нормализованный выход: 0.6225

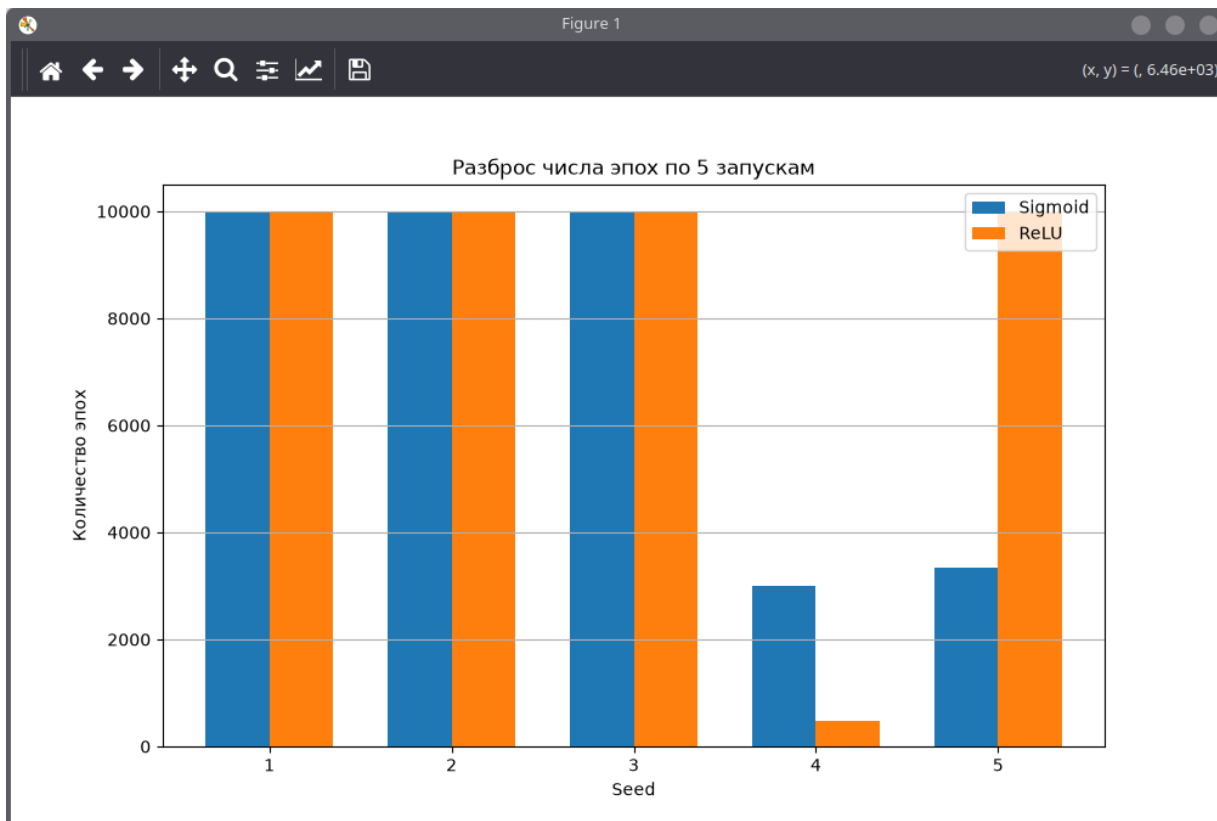
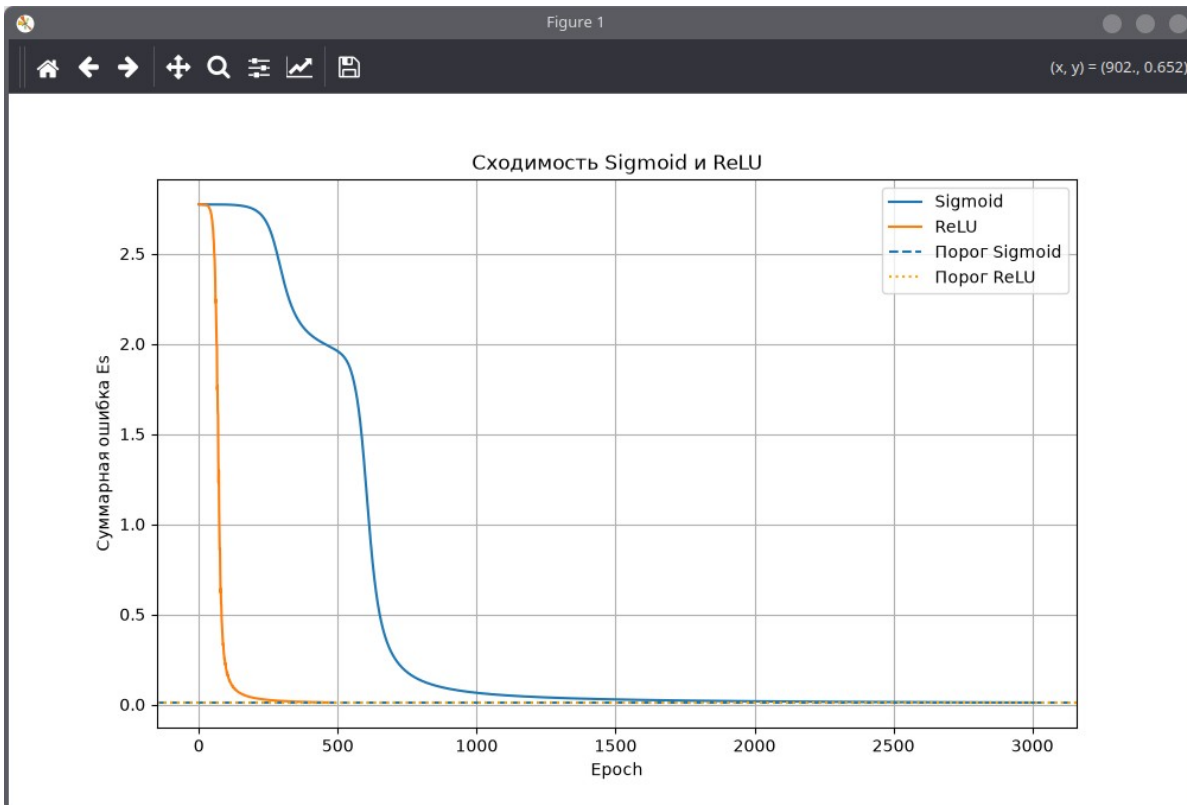
Выход в исходной шкале: 0.8877

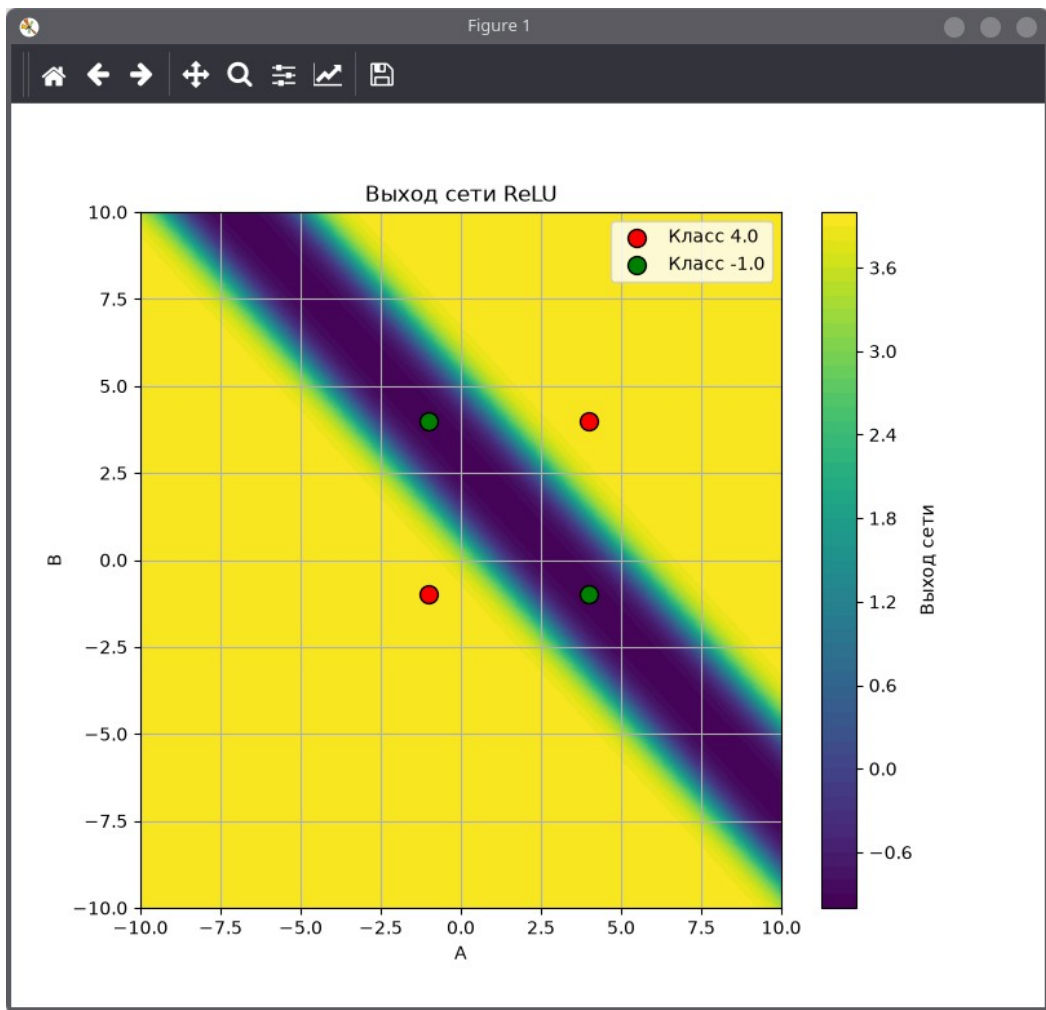
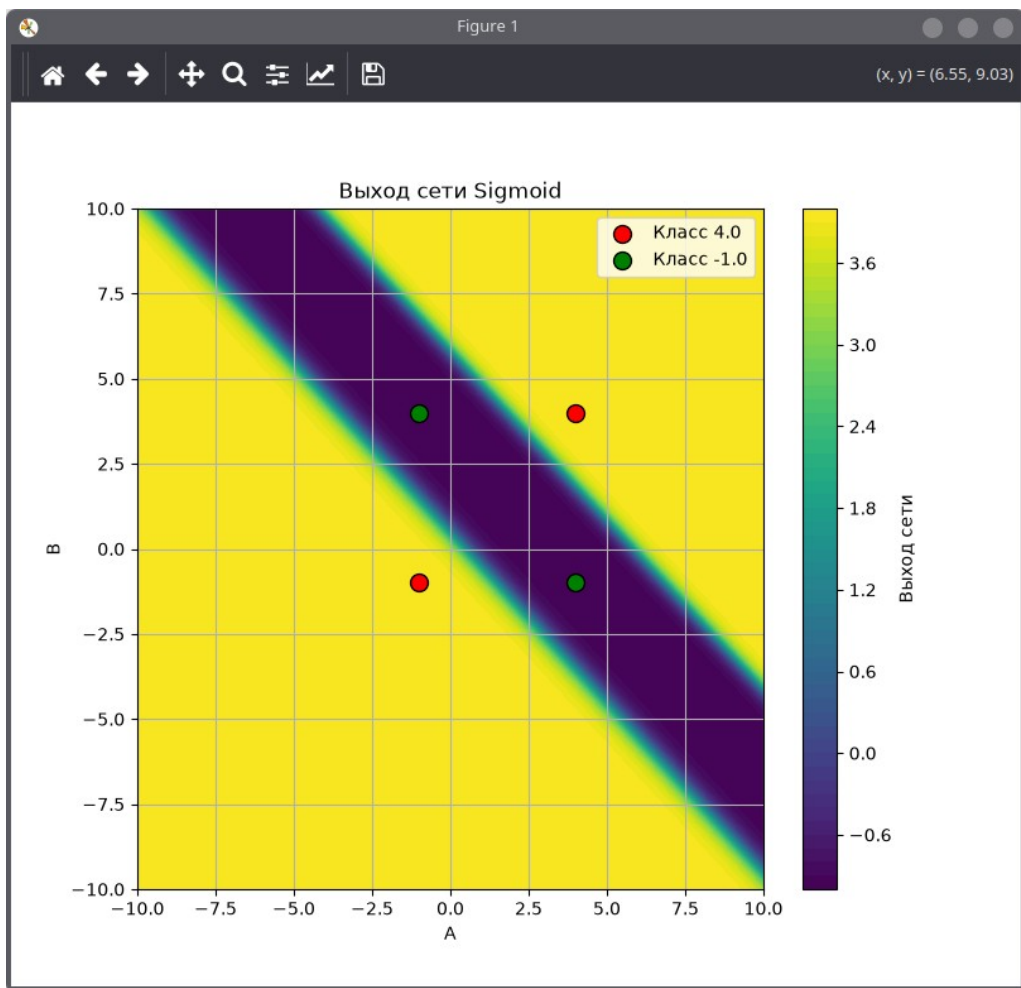
Ближайший класс: -1

Введите пару чисел (A, B): 0 5

Нормализованный выход: 0.6292
Выход в исходной шкале: 0.8541
Ближайший класс: -1

Введите пару чисел (A, B): 0 0
Нормализованный выход: 0.1189
Выход в исходной шкале: 3.4054
Ближайший класс: 4





Вывод:

1. Какая из активационных функций (Сигмоида или ReLU) обеспечила более быструю сходимость? Более устойчива?

В данном эксперименте **Sigmoid** оказалась более устойчивой. При $\alpha = 0.4$, одинаковой архитектуре 2–2–1 и одинаковом критерии $E_e = 0.01$ она успешно обучилась при большем числе начальных инициализаций.

Для успешного запуска с $\text{seed}=4$:

- Sigmoid – около **3010** эпох;
- ReLU – **488** эпох.

То есть в случае удачной инициализации **ReLU** сошла **значительно быстрее**, но ReLU оказалась более чувствительной к начальным весам. При других seed она могла остановиться на ошибке 1.39–1.92 и не достичь заданного критерия даже за 10000 эпох.

2. Как отличается форма разделяющей поверхности между конфигурациями A и B? Наблюдаются ли ожидаемые кусочно-линейные границы для ReLU?

Для Sigmoid граница получается гладкой, поскольку сама функция активации непрерывно изменяет значение нейрона.

Для ReLU скрытые нейроны создают **кусочно-линейную поверхность**: в областях, где $S1 \leq 0$, соответствующий нейрон выдаёт строго 0, а при $S1 > 0$ его выход изменяется линейно.

Поэтому для ReLU действительно наблюдаются ожидаемые кусочно-линейные границы. В двумерном случае это проявляется как комбинация линейных участков.

3. Почему ReLU часто предпочитают Сигмоиде в глубоких сетях? Подтверждается ли это на XOR?

ReLU обычно предпочтительнее в глубоких сетях потому, что при положительных значениях её производная равна 1. Это уменьшает проблему затухания градиента по сравнению с Sigmoid, у которой производная не превышает 0.25 и становится очень маленькой при насыщении.

4. Изменилась ли способность сети решать XOR при переходе на ReLU, или оба варианта успешно справляются благодаря архитектуре?

Нет. При удачной инициализации обе функции способны решить XOR.

Это подтверждается результатами с $\text{seed}=4$: обе конфигурации достигли примерно 100% accuracy и ошибки около 0.01.

5. Есть ли случаи "мёртвых нейронов" при ReLU (нейронов, выдающих 0)? Как это повлияло на обучение?

В неудачном запуске (Random_seed = 42) наблюдалось состояние, когда оба скрытых нейрона одновременно выдавали ноль для некоторых входов:

$$[0,1] \rightarrow h = [0, 0]$$

$$[1,1] \rightarrow h = [0, 0]$$

$$S1 = [-0.3402, -3.4355] \text{ ReLU}'(S1) = [0, 0]$$

$$S1 = [-2.8640, -0.5351] \text{ ReLU}'(S1) = [0, 0]$$

Поэтому для этих образцов градиент скрытого слоя равен нулю. Это затрудняет изменение весов скрытого слоя и приводит к застою обучения.

6. Как выбор активационной функции соотносится с необходимостью многослойной архитектуры для XOR?

Для рассматриваемых функций Sigmoid и ReLU наличие скрытого слоя необходимо, поскольку одна линейная комбинация входов не позволяет корректно разделить классы XOR. Скрытый слой с нелинейной функцией активации позволяет сформировать нелинейную разделяющую поверхность и правильно классифицировать все четыре точки.

7. Что показывает визуализация разделяющей поверхности для обеих конфигураций? Совпадает ли граница между классами с ожидаемой для XOR?

Визуализация показывает, как сеть разделяет четыре точки XOR на два класса. Точки одного класса расположены на одной диагонали, а точки другого — на противоположной.